Abilità Informatiche 2026/27

L04 · Algoritmi e pensiero computazionale

Andrea Schimmenti · Università di Bologna

Di che cosa parliamo oggi

1. Il **pensiero computazionale**
2. Che cos'è un **algoritmo** (e una ricetta per un robot)
3. Come si **scrive** un algoritmo: linguaggio naturale, pseudocodice, diagrammi, Python
4. Le **strutture di controllo**: sequenza, selezione, iterazione
5. **Ordinare** e **cercare** nelle diverse lingue
6. "L'algoritmo" nel **discorso pubblico**

1. Il pensiero computazionale

Da Papert a Wing

- Anni '80: **Seymour Papert** (MIT, linguaggio Logo) usa l'espressione per i bambini che imparano programmando
- **2006**: **Jeannette Wing** la propone come abilità **per tutti**, accanto a leggere, scrivere e far di conto
- Un insieme di strategie per affrontare un problema in modo che la soluzione possa essere eseguita, almeno in parte, da una **macchina**

Wing, J. M. (2006). Computational thinking. Communications of the ACM, 49(3), 33–35. https://doi.org/10.1145/1118178.1118215

I quattro principi

Principio	Che cosa significa	Esempio linguistico
Scomposizione	dividere in sotto-problemi	romanzo → capitoli → paragrafi → frasi
Schemi	trovare regolarità	i verbi in *-are* si coniugano allo stesso modo
Astrazione	tenere solo ciò che conta	per il plurale conta la vocale finale, non il significato
Algoritmo	scrivere una procedura precisa	le istruzioni passo per passo per il plurale

Che cosa non è

- Non significa "pensare come un computer"
- Non tutti i problemi hanno una soluzione algoritmica
- Per Papert il computer era uno strumento per **imparare ed esprimersi**, non un modello della mente

Interpretare una metafora, tradurre un gioco di parole: non c'è ragione di forzarli in uno schema.

Lodi, M., & Martini, S. (2021). Computational thinking, between Papert and Wing. Science & Education, 30(4), 883–908. https://doi.org/10.1007/s11191-021-00202-5

2. Che cos'è un algoritmo

Una definizione

Dal nome di **al-Khwārizmī** (Baghdad, IX secolo): *Dixit Algorizmi*.

Un **algoritmo** è una sequenza **finita** di istruzioni **precise e non ambigue** che, da un **input**, produce un **output** in un numero finito di passi.

- **Finitezza**: termina
- **Non ambiguità**: ogni istruzione ha un solo significato
- **Eseguibilità**: ogni passo si può fare davvero
- **Generalità**: vale per un'intera classe di problemi

Algoritmi quotidiani

- Provare le scarpe: la mia taglia → stretta? quella più grande → larga? quella più piccola → mi fermo quando va bene
- Montare un mobile seguendo le istruzioni
- Cercare una parola nel vocabolario

E la **ricetta**? È un algoritmo **solo in parte**:

> "Aggiungete sale **quanto basta**"

Quanto basta lo decide il cuoco. Un computer **non ha buon senso**: fa esattamente ciò che c'è scritto.

Esercizio 1: la ricetta per un robot (15 minuti)

"*Cuocete 320 g di spaghetti in abbondante acqua salata.*" Per un robot diventa:

1. prendi una pentola capiente
2. riempila d'acqua fino a tre quarti
3. mettila sul fornello più grande
4. accendi il fornello alla fiamma massima
5. aspetta finché l'acqua non bolle
6. aggiungi 10 g di sale
7. ...

Continuate voi, con **azioni elementari**. Un compagno fa il robot e segnala ogni ambiguità.

Come si dice *al dente*? E se il fornello non si accende?

Programma e linguaggio di programmazione

- **Programma** = algoritmo scritto in un **linguaggio di programmazione** (Python, JavaScript, R...)
- I linguaggi di programmazione sono **linguaggi formali**: vocabolario limitato, grammatica rigida, **nessuna ambiguità**
- Una parentesi non chiusa non viene "capita lo stesso": il programma **non funziona**

> "Ho visto l'uomo con il cannocchiale" · "La vecchia porta la sbarra"

Le lingue naturali sono piene di **ambiguità**: il cuore del problema del NLP *(Blocco C)*.

3. Come si scrive un algoritmo

Il plurale: linguaggio naturale e pseudocodice

> Guarda l'ultima lettera. Se è *-o* → *-i*. Se è *-a* → *-e*. Se è *-e* → *-i*. Altrimenti lascia la parola invariata.

```
ALGORITMO plurale(parola)
    SE parola finisce con "o" ALLORA
        restituisci parola senza l'ultima lettera + "i"
    ALTRIMENTI SE parola finisce con "a" ALLORA
        restituisci parola senza l'ultima lettera + "e"
    ALTRIMENTI SE parola finisce con "e" ALLORA
        restituisci parola senza l'ultima lettera + "i"
    ALTRIMENTI
        restituisci parola          // "film", "bar", "città"
FINE
```

Lo **pseudocodice** non ha una sintassi ufficiale: conta la **precisione**.

Il diagramma di flusso

- **Ovali**: inizio e fine
- **Rettangoli**: azioni
- **Rombi**: decisioni (sì / no)
- **Frecce**: l'ordine

```
( Inizio: parola )
        |
< finisce in -o? > --sì--> [ -o → -i ] --> ( Fine )
        | no
< finisce in -a? > --sì--> [ -a → -e ] --> ( Fine )
        | no
< finisce in -e? > --sì--> [ -e → -i ] --> ( Fine )
        | no
[ lascia invariata ] --> ( Fine )
```

Lo stesso algoritmo in Python

```
def plurale(parola):
    if parola.endswith("o"):
        return parola[:-1] + "i"
    elif parola.endswith("a"):
        return parola[:-1] + "e"
    elif parola.endswith("e"):
        return parola[:-1] + "i"
    else:
        return parola

print(plurale("libro"))   # libri
print(plurale("casa"))    # case
print(plurale("film"))    # film
```

`def` definisce una funzione · `if` / `elif` / `else` = se / altrimenti se / altrimenti · `parola[:-1]` = senza l'ultimo carattere · `#` = commento

Mettiamolo alla prova

Parola	Risultato	
libro, casa, fiore, film	libri, case, fiori, film	✅
città	città	✅ (*à* ≠ *a*!)
mano	mani	✅ per caso: *mano* è femminile
amica	amice	❌ amiche
lago	lagi	❌ laghi
problema	probleme	❌ problemi
uomo	uomi	❌ uomini
uovo	uovi	❌ uova

Regole, eccezioni, eccezioni delle eccezioni

- *-ca* → *-che*, *-ga* → *-ghe*… ma *amico* → *amici*, *antico* → *antichi*
- *mago* → *maghi*, *psicologo* → *psicologi*
- *-ma* greco → *-mi* (*problema*, *tema*)… ma *arma* → *armi*, *mamma* → *mamme*

Alla fine serve un **elenco di eccezioni**, che cresce a ogni testo nuovo.

È il problema della prima **traduzione automatica**. Dagli anni '90: invece di scrivere le regole, le si fa **imparare da milioni di esempi** *(L10, L12)*.

4. Le strutture di controllo

Tre ingredienti per qualsiasi algoritmo

Sequenza

fai A, poi B, poi C

(la ricetta per il robot)

Selezione

se la condizione è vera fai A, altrimenti B

(il plurale)

Iterazione

ripeti finché la condizione resta vera

(aspetta finché l'acqua non bolle)

Variabili

contenitori con un nome, il cui contenuto cambia

Un esempio con tutte e tre: contare le parole

```
ALGORITMO conta_parole(testo)
    frequenze ← dizionario vuoto
    PER OGNI parola IN testo                    // iterazione
        parola ← minuscolo(parola)              // sequenza
        SE parola è già in frequenze ALLORA     // selezione
            frequenze[parola] ← frequenze[parola] + 1
        ALTRIMENTI
            frequenze[parola] ← 1
    restituisci frequenze
```

```
frequenze = {}
for parola in testo.lower().replace(",", "").split():
    if parola in frequenze:
        frequenze[parola] += 1
    else:
        frequenze[parola] = 1
```

Due osservazioni

1. Il programma usa un **dizionario** (L03): parola → numero di occorrenze
2. Tratta *quest'ermo* come **una sola parola**, perché divide solo agli spazi

Decidere che cosa sia una "parola" è meno semplice di quanto sembri: è la **tokenizzazione** *(L09)*.

Volete provare? Google Colab o JupyterLite, direttamente nel browser, senza installare nulla.

5. Ordinare e cercare

Che cosa vuol dire "in ordine alfabetico"?

- **Spagnolo**: *ñ* dopo *n* (*nuez*, *ñandú*, *oso*); fino al 1994 *ch* e *ll* erano lettere a sé
- **Svedese**: *å*, *ä*, *ö* **dopo la z**
- **Tedesco**: *ä* come *a* nei dizionari, come *ae* negli elenchi di nomi
- **Ceco**: *ch* è una lettera, dopo la *h*
- **Giapponese**: ordine del sillabario (*gojūon*)
- **Cinese**: per *pinyin*, per numero di tratti, per radicale

Per un computer senza istruzioni: `Z` (90) < `a` (97) < `z` (122) < `é` (233)

zebra, Zurigo, école, abete → **Zurigo, abete, zebra, école**

Servono regole per ogni lingua: la *collation*.

Ordinare: ricordate la fila della L01?

Strategia	Come funziona
Bubble sort	ci si confronta con il vicino e ci si scambia se nell'ordine sbagliato
Selection sort	si cerca il più anziano e lo si mette in testa, poi il successivo…
Insertion sort	ognuno entra nella fila già formata al posto giusto (come le carte in mano)

Tutte corrette, ma con **costi molto diversi** al crescere delle persone.

Merge sort e *quicksort* usano la **scomposizione**: dividono, ordinano le metà, le fondono.

Cercare in un dizionario

Ricerca lineare: pagina dopo pagina dalla prima.

Ricerca binaria: apri a metà, scarti metà, ripeti.

Voci	Lineare (caso peggiore)	Binaria (caso peggiore)
1.000	1.000 passi	10 passi
1.000.000	1.000.000 passi	20 passi
1.000.000.000	1.000.000.000 passi	30 passi

La ricerca binaria funziona **solo se l'elenco è ordinato**.

Non basta sapere se un problema si risolve: conta **quanto costa** (tempo, memoria, energia).

6. "L'algoritmo" nel discorso pubblico

Quando i giornali dicono "l'algoritmo"

- Non una procedura leggibile riga per riga come il plurale
- Un **sistema**: molti programmi + **modelli statistici** addestrati sui dati degli utenti
- Spesso le regole **non si possono leggere**: segreto industriale, oppure ricavate dai dati e mai scritte

Per discutere: se un sistema decide quali notizie vedete, chi è responsabile? Chi l'ha progettato? Chi l'ha addestrato? Voi che avete cliccato?
Se un algoritmo esclude la vostra candidatura, avete diritto di sapere **perché**? *(L07, L15)*

Attività (in stanze da 3–4)

1. **La ricetta per il robot**: riscrivetela come sequenza di azioni elementari; un compagno la esegue alla lettera
2. **Il plurale nella vostra lingua**: pseudocodice per il plurale (o il genere, il participio, il comparativo). Provatelo su **10 parole**, almeno **3 difficili**. Quante regole? Quante eccezioni? Vi serviva sapere qualcosa oltre alla forma?
3. **Disegnatelo**: diagramma di flusso su carta o con *draw.io*

Riassumendo

- **Pensiero computazionale**: scomposizione, schemi, astrazione, algoritmi
- **Algoritmo**: finito, preciso, eseguibile, generale. Una ricetta lo è solo in parte
- Si scrive in **linguaggio naturale**, **pseudocodice**, **diagrammi di flusso**, **Python**
- **Sequenza**, **selezione**, **iterazione** + **variabili**
- Le lingue sfuggono alle regole: **eccezioni** → apprendimento dai dati
- L'**ordine alfabetico** dipende dalla lingua; la **ricerca binaria** richiede un elenco ordinato
- "L'algoritmo" dei giornali è un **sistema**, spesso opaco

Prossima lezione

L05 · Caso di studio: una bibliografia come base di dati

Tutorial con Zotero